

A Novel Algorithm for Finding the Optimal Solution for a Time-Minimizing Solid Transshipment Problem

Archana Khurana, Niko Zhang, Fei Yuan

Abstract

This paper presents a novel algorithm for solving Time-Minimizing Solid Transshipment Problems (TMSTsP). The TMSTsP is three-dimensional, involving multiple commodity types and additional constraints on transshipped quantities, and is relevant for managers seeking to optimize the total transshipment time. This is useful for the emergency department during a widespread natural disaster, when the top priority is saving as many lives with limited equipment. The proposed algorithm minimizes the total quantity of commodities transshipped along maximum-duration pathways by transforming the problem into an equivalent time-minimizing solid transportation problem. The algorithm provides an optimal solution in a finite number of iterations under the limitation that all goods are transported simultaneously in a single trip. We also present a systematic approach for identifying a multi-loop that introduces a nonbasic cell into the basis, yielding an improved basic feasible solution. We pseudo-randomly generate balanced problems ranging from 4 to 80 total transshipment nodes and implemented the algorithm in Python. The algorithm's number of iterations, total allocations at maximum-duration pathways, and runtime are compared to those of Simplex using the CPLEX solver in GAMS. The computational results demonstrate that the number of iterations for our algorithm is fewer than that of the Simplex method for problems with the same number of allocations. Moreover, we show that our algorithm has fewer allocations for routes taking maximum time, which conserves resources and reduces the probability of delays.

Keywords: Solid, Multi-index, Transshipment, Transportation, Time-minimizing, Multi-loop

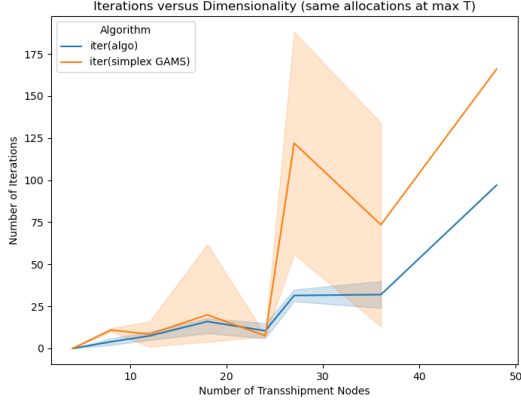
Computational Results

We implemented and tested our algorithm using Python on a local machine equipped with an Nvidia GeForce RTX 3080 Ti laptop GPU. Our code generates pseudo-random balanced TMSTsP instances with dimensions a, b, e that specified by the user, as described in the previous section. Additionally, the code produces the corresponding GAMS code for the generated problem.

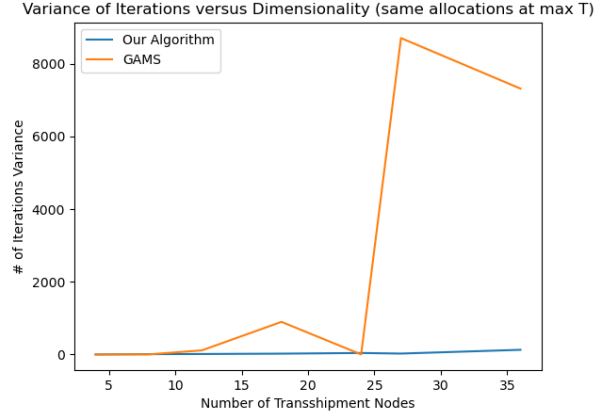
5 trials were conducted for each of the following problem dimensions: $(2 \times 2 \times 1)$, $(2 \times 2 \times 2)$, $(2 \times 2 \times 3)$, $(2 \times 2 \times 4)$, $(2 \times 2 \times 5)$, $(2 \times 3 \times 2)$, $(2 \times 3 \times 3)$, $(2 \times 3 \times 4)$, $(2 \times 3 \times 5)$, $(3 \times 3 \times 2)$, $(3 \times 3 \times 3)$, $(3 \times 3 \times 4)$, $(3 \times 4 \times 2)$, $(3 \times 4 \times 3)$, $(3 \times 4 \times 4)$, $(4 \times 4 \times 3)$, $(4 \times 4 \times 4)$, and $(4 \times 4 \times 5)$.

In each trial, a problem of dimensions $(a \times b \times e)$ was generated and solved using our implementation. Subsequently, the same problem was solved using GAMS, which employs the CPLEX solver, for verification. CPLEX utilizes the simplex method on the LP relaxations within its branch-and-bound framework to solve the mixed-integer programming (MIP) problem. The number of iterations, maximum time T , allocations counts at time T (for both GAMS and our implementation), and execution times were recorded in a CVS file.

E-mail addresses: akhurana@umd.edu (A. Khurana), hiniko23@gmail.com (N. Zhang), yuanfei20160811@gmail.com (F. Yuan)

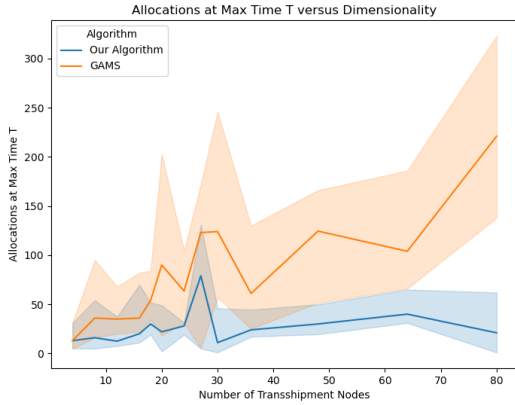


(a) Median Number of Iterations vs Dimensionality with 95% confidence interval

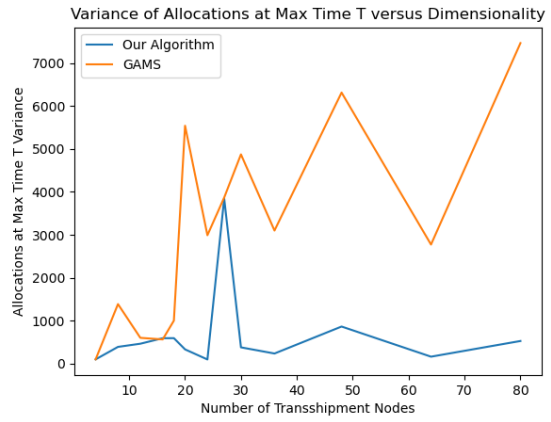


(b) Variance of Number of Iterations vs Dimensionality

Figure 1: Sub-figure (a) indicates that for problems where both GAMS and our algorithm yielded identical allocations for commodities with a shipment time equal to T , our algorithm required fewer iterations than GAMS. However, both algorithms appear to exhibit exponential runtime behavior. Sub-figure (b) displays the variance of the data in Sub-figure (a); notably, GAMS demonstrates a larger variance compared to our algorithm. A Wilcoxon signed-rank test comparing the average iterations for GAMS and our algorithm indicated that the proposed algorithm requires significantly fewer iterations than CPLEX ($p = 1.2127\text{e-}13$).

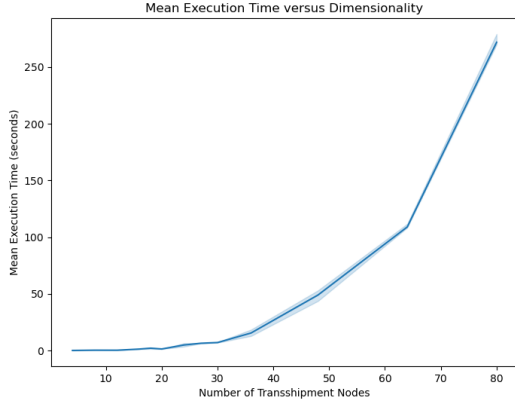


(a) Median Top Allocations vs Dimensionality with 95% confidence interval

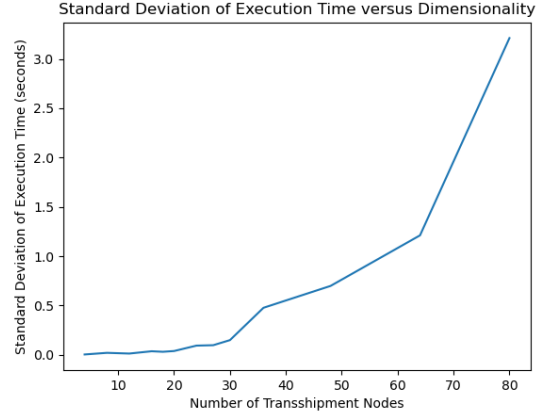


(b) Variance of Top Allocations vs Dimensionality

Figure 2: Sub-figure (a) demonstrates that as the problem dimension increases, our algorithm yields solutions with minimum allocations for commodities having a shipment time equal to T , maintaining relative stability. In contrast, GAMS produces allocations that increase exponentially with the problem's dimensionality. Sub-figure (b) illustrates the variance of the data presented in Sub-figure (a); notably, GAMS exhibits a larger variance compared to our algorithm. A Wilcoxon signed-rank test comparing the average allocations for GAMS and our algorithm indicated that the proposed algorithm has significantly fewer allocations than CPLEX for transshipment routes taking the most amount of time ($p = 0.0367$).



(a) Mean Execution Time versus Dimensionality with 95% confidence interval



(b) SD of Execution Time versus Dimensionality

Figure 3: Sub-figure (a) indicates that our algorithm’s mean execution time grows exponentially with respect to the problem’s dimensionality. Similarly, Sub-figure (b) demonstrates that the standard deviation of the execution time also exhibits exponential growth relative to the problem’s dimensionality. Each data point is calculated from five trails.

Conclusions and Future Scope

We have developed a mathematical formulation and a novel algorithm for solving the Time-Minimizing Solid Transshipment Problem (TMSTsP). The algorithm transforms the TMSTsP into an equivalent time-minimizing solid transportation problem and solves it by minimizing the quantity of commodities transshipped along the maximum-duration pathways over a finite number of iterations. It provides advantages such as minimizing resource utilization and reducing associated penalties, including fuel expenses and worker fatigue.

The algorithm assumes that there is sufficient capacity to transport commodities from sources to sinks in a single trip and that all goods are transported simultaneously. Additionally, we propose a systematic approach to identify multi-loops for introducing a nonbasic cell into the basis, thereby improving the basic feasible solution.

To evaluate the performance of our algorithm, we compared it with the solver that uses the Simplex method by solving 18 instances of varying dimensions, ranging from $(2 \times 2 \times 1)$ to $(4 \times 4 \times 5)$. Our algorithm was implemented in Python, while the Simplex method was executed in GAMS using CPLEX solver. The results demonstrate that, on average, our algorithm yields fewer allocations along maximum-duration pathways compared to the Simplex method. For instances where both methods result in the same number of such allocations, our algorithm requires fewer iterations on average. Lastly, the proposed algorithm is shown to also minimize allocations for transshipment routes taking up the most amount of time, something that CPLEX can not do in conjunction with time minimization.

For future research, we aim to extend our approach to higher-dimensional problems and to enhance the efficiency of identifying multi-loops, which play a critical role in multi-index transportation problems.